

WAMMETKA

Handoff técnico y funcional del MVP local

Estado del proyecto

Se construyó un MVP local de marketplace regional para La Guajira. El recorrido del comprador puede probarse de punta a punta: catálogo, carrito, dirección, creación de pedido, pago simulado, flujo operativo, entrega, calificación y PQR. También existe un centro interno de operaciones para consultar los pedidos.

Arquitectura implementada

Capa	Implementación actual
API	NestJS + TypeScript; monolito modular en wammetka-api.
Web	React/Vinext en wammetka-web; interfaz de comprador y centro de operaciones.
Persistencia	Archivo JSON local de desarrollo: wammetka-api/data/wammetka-state.json.
Diseño	Glassmorphism inspirado en visionOS, con acento naranja Wammetka y banner promocional.
Entorno	API local en puerto 3000; interfaz local en puerto 3001.

Módulos construidos y validados

- **Identidad y roles:** registro, inicio de sesión, comprador, comercio, repartidor, soporte, operaciones, finanzas y administrador.
- **Cobertura:** municipios piloto y zonas de cobertura configurables.
- **Comercios y catálogo:** comercio, sucursal, producto, precio y stock; catálogo filtrado por municipio.
- **Checkout:** carrito de un comercio, dirección o referencia de entrega y reserva de inventario.
- **Pagos:** tarjeta de prueba y efectivo contra entrega; transición a pendiente de aceptación.
- **Operación logística:** aceptación, preparación, listo para recoger, en ruta y entregado, con reglas de transición.
- **Postventa:** calificación de 1 a 5 estrellas y comentario, solo después de entrega.
- **Soporte/PQR:** casos de pedido, pago, entrega, producto u otros, persistidos localmente.
- **Centro de operaciones:** listado interno de pedidos con producto, dirección, total y estado.

Cómo ejecutar y probar

1. Iniciar la API

```
C:\Users\USUARIO\.cache\codex-runtimes\codex-primary-runtime\dependencies\node\bin\node.exe
.\node_modules\tsx\dist\cli.mjs src\main.ts
```

Ejecutar desde la carpeta wammetka-api.

2. Iniciar la interfaz

```
C:\Users\USUARIO\.cache\codex-runtimes\codex-primary-runtime\dependencies\node\bin\node.exe
.\node_modules\vinext\dist\cli.js dev
```

Ejecutar desde la carpeta wammetka-web. Abrir <http://localhost:3001>.

Recorrido de prueba

- Actualizar catálogo y agregar un producto real.
- Ingresar una referencia de entrega y crear el pedido.
- Elegir tarjeta de prueba o efectivo contra entrega; confirmar pago.
- Avanzar por los botones de simulación operativa hasta Entregado.
- Enviar una calificación y, si se requiere, abrir Ayuda para registrar una PQR.
- Abrir Operación para consultar los pedidos persistidos.

Rutas API relevantes

Uso	Ruta
Salud / índice	GET /api/v1/health y GET /api/v1
Catálogo	GET /api/v1/catalog/products?municipality=Fonseca
Pedido	POST /api/v1/orders; GET /api/v1/orders/:orderId
Pago	POST /api/v1/orders/:orderId/payments
Estados	POST /api/v1/orders/:orderId/status
Calificación	POST /api/v1/orders/:orderId/rating
Operación	GET /api/v1/admin/orders
PQR	POST /api/v1/support/cases; GET /api/v1/support/cases/admin

Verificación realizada

- Compilaciones TypeScript y builds de la interfaz completados exitosamente.
- Pruebas automáticas para identidad, cobertura, comercios, catálogo, pedidos, reserva de stock, pago, transición de estado y calificación.
- Pruebas manuales de interfaz: actualización de catálogo, pedido, tarjeta de prueba, flujo hasta Entregado, calificación, banner, glassmorphism y centro de operaciones.

Limitaciones actuales

- La persistencia JSON es adecuada solo para desarrollo local; no reemplaza PostgreSQL, backups, concurrencia ni auditoría productiva.
- No hay Docker ni PostgreSQL instalados en el equipo al momento del handoff.
- Pagos, aceptación de comercio y logística son simulaciones de prueba; no hay pasarela, mapas, GPS, notificaciones ni app de repartidor real.
- Los endpoints dev/seed-* existen para facilitar pruebas locales y deben eliminarse o restringirse antes de producción.
- La dirección actual es texto/referencia; falta geolocalización, zonas tarifarias reales y evidencia de entrega.

Próximos pasos recomendados

- Instalar Docker/PostgreSQL y migrar el estado JSON a PostgreSQL mediante repositorios y migraciones.
- Implementar autenticación segura con JWT, verificación telefónica y permisos por recurso.
- Crear portal real de comercio y app de repartidor; retirar simulaciones operativas del flujo de comprador.
- Integrar pasarela de pagos colombiana, mapas/geocodificación, notificaciones y almacenamiento de evidencias.
- Agregar tarifas por zona, asignación de repartidor, conciliación, liquidaciones y tablero de PQR.
- Realizar revisión jurídica, privacidad, términos, protección del consumidor y pruebas de campo antes del piloto.

Ubicaciones principales

API: C:\Users\USUARIO\Documents\Codex\2026-08-22\pdf-plugin-pdf-openai-primary-runtime-4\wammetka-api

Interfaz: C:\Users\USUARIO\Documents\Codex\2026-08-22\pdf-plugin-pdf-openai-primary-runtime-4\wammetka-web

Estado local: wammetka-api/data/wammetka-state.json